Reporte Técnico de Actividades Práctico-Experimentales Nro. 006

1. Datos de Identificación del Estudiante y la Práctica

Nombre del estudiante	Anderson Mateo Coello Jaramillo
Asignatura	Simulación
Ciclo	5
Unidad	U2: Generación de números pseudoaleatorios y generación de variables aleatorias
Resultado de aprendizaje de la unidad	R2. Interpreta los resultados de una Simulación, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	006
Título de la Práctica	Validación estadística de números pseudoaleatorios mediante Chi-cuadrado y Kolmogorov-Smirnov
Nombre del Docente	José O. Guamán Q.
Fecha de la práctica	05 de noviembre de 2026 06 de noviembre de 2026
Horario	Jueves 10h30 a 11h30 Viernes 07h30 a 09h30
Lugar	Aula
Tiempo planificado en el Sílabo	3 horas

2. Objetivo(s) de la Práctica

Evaluar la calidad estadística de una secuencia de números pseudoaleatorios mediante Chi-cuadrado y Kolmogorov-Smirnov para determinar si su comportamiento es compatible con una distribución uniforme en [0,1].

1. Generar secuencias mediante el método congruencial lineal (LCG).

2. Aplicar Chi-cuadrado para evaluar la uniformidad de las frecuencias.

3. Aplicar KS para comparar la distribución empírica con la uniforme teórica.

4. Interpretar los estadísticos y sus respectivos valores p.

5. Comparar diferentes configuraciones del generador pseudoaleatorio.

3. Materiales y reactivos

Guía de Actividades Práctico-Experimentales Nro. 006 [1], diapositivas de las semanas 5 y 6 [2, 3], configuración compartida **config/generadores.csv** y código de las implementaciones Python y Java [4]. Se trabajó con secuencias producidas por la recurrencia LCG; no se sustituyeron por otro generador aleatorio.

4. Equipos y herramientas

Herramienta	Versión / uso comprobado
Computador y sistema	Linux-7.0.0-42-generic-x86_64-with-glibc2.43
Python	3.14.4; entorno virtual python/.venv
Java	JDK 25.0.5; compilación y pruebas con Maven
Cálculo y tablas	NumPy 2.5.4, SciPy 1.18.1 y Pandas 3.1.1
Gráficas e informe	Matplotlib 3.11.2, Java2D y ReportLab 5.1.0
Verificación	pytest 9.2.0 y JUnit 5 (configurado en pom.xml)

Las versiones de Python, Java y sistema proceden de los archivos de entorno del benchmark; las de las bibliotecas Python corresponden al entorno usado para generar este informe. No se atribuyen al equipo especificaciones de hardware que no estén documentadas.

5. Procedimiento / Metodología Ejecutada

5.1. Generador congruencial lineal y parámetros

Se asigna la semilla X_0 una sola vez. En cada iteración se obtiene el nuevo estado entero y después se normaliza. El primer número almacenado corresponde a X_1 [1, 2].

$$X_{n+1} = (aX_n + c) \bmod m \qquad U_{n+1} = X_{n+1} / m$$

Configuración	Semilla X_0	a	c	m
MINSTD	123457	16807	0	2147483647
glibc	1	1103515245	12345	4294967296
ZX81	1	75	74	131073

Los parámetros MINSTD y glibc, incluidas sus semillas, se tomaron de los ejemplos de clase [2, 3]. Los parámetros a, c y m de ZX81 corresponden a la tabla de Semana 5; su semilla 1 es una elección del experimento. Los nombres identifican las configuraciones de clase, sin afirmar que reproduzcan toda la API de los generadores históricos.

5.2. Hipótesis y nivel de significancia

Para las pruebas de uniformidad se plantean H_0: los valores siguen U(0,1), y H_1: su distribución difiere de U(0,1). Se fijó **$\alpha = 0.05$**. Si $p < \alpha$ se rechaza H_0; si $p \geq \alpha$, no se rechaza. No rechazar no equivale a demostrar que H_0 sea verdadera [1, 3].

5.3. Prueba Chi-cuadrado

Se dividió [0,1) en diez intervalos de amplitud 0.1. Para cada uno se contabilizó la frecuencia observada O_i y se calculó $E_i = N/10$. Con N = 10 000, $E_i = 1000$ y los grados de libertad son $gl = 10 - 1 = 9$. Se satisface la condición $E_i \geq 5$.

$$\chi^2 = \Sigma_{i=1,\ldots,10} (O_i - E_i)^2 / E_i$$

El valor p corresponde a la cola superior de χ^2 con nueve grados de libertad. Python utiliza SciPy y Java la función gamma regularizada Q de Apache Commons Math. El crítico a $\alpha = 0.05$ es 16.91897760. Se exportan las frecuencias y los aportes al estadístico.

5.4. Prueba Kolmogorov-Smirnov bilateral

Se ordenó una copia de los datos para conservar la secuencia original. Con $U_{(i)}$ ordenados e i = 1,…,N, se calcularon ambas diferencias respecto a F(u) = u:

$$D^+ = \max(i/N - U_{(i)})$$
$$D^- = \max(U_{(i)} - (i-1)/N)$$
$$D = \max(D^+, D^-)$$

En ambos lenguajes se usó la distribución límite bilateral de Kolmogorov: $p \approx Q_K(\sqrt{N} \cdot D)$. Es un **valor p asintótico**, no exacto para N finito. El crítico obtenido es 0.01358099; la aproximación de clase $1.36/\sqrt{N}$ da 0.0136. Se guardaron los 10 000 datos ordenados, las FDE izquierda/derecha y las diferencias.

5.5. Prueba de rachas como complemento

Se clasificaron los valores por encima y por debajo de la mediana, manteniendo el orden original. Se eliminarían los empates, informando su número. La hipótesis de referencia es el orden aleatorio de esos signos; difiere de la hipótesis de uniformidad [3].

$$\mu_R = 2n_1n_2/(n_1+n_2) + 1$$
$$\sigma^2_R = 2n_1n_2(2n_1n_2 - n_1 - n_2) / [(n_1+n_2)^2(n_1+n_2-1)]$$
$$Z = (R - \mu_R)/\sigma_R$$

Se usa la aproximación normal bilateral cuando $n_1, n_2 > 10$, sin corrección de continuidad. El crítico es $|Z| \approx 1.95996$. La prueba se añadió como complemento de Semana 6; no sustituye Chi-cuadrado ni KS.

5.6. Arquitectura e implementación en Python y Java

Componente	Responsabilidad implementada
Modelo	LCG, parámetros, resultados, pruebas estadísticas, estimación de π y medición del rendimiento.
Vista	Presentación en consola y gráficas: Matplotlib en Python y Java2D en Java.
Controlador	Coordina generación, análisis, exportación y experimentos de rendimiento.
Infraestructura	Lectura de configuración, persistencia CSV/JSON y comparación entre lenguajes.
Strategy	PruebaEstadistica define el contrato de Chi-cuadrado, KS y rachas; el servicio recibe las estrategias.

Los modelos no importan vistas ni controladores. Se proporcionan dependencias por constructor. Java ejecuta sus cálculos y crea sus archivos y gráficas sin invocar Python. La comparación consolidada se realiza después de ambas ejecuciones. Java usa enteros long y recurre a BigInteger cuando los parámetros podrían desbordar el producto; Python realiza la recurrencia con enteros de precisión arbitraria.

5.7. Secuencia de ejecución y comparación

1. Leer los mismos parámetros del archivo config/generadores.csv en ambos programas.

2. Generar 10 000 valores por configuración y aplicar las tres pruebas con $\alpha = 0.05$.

3. Exportar secuencias, frecuencias, datos KS, resúmenes y gráficas por lenguaje.

4. Comparar los 30 000 estados y valores normalizados, comprobando también la recurrencia y las decisiones.

5. Ejecutar la estimación de π para los cuatro tamaños y consolidar las doce filas de la tabla solicitada.

Los estados enteros y normalizados se comparan con igualdad exacta. Para estadísticos y valores p se aplican tolerancias absoluta y relativa de 10^{-10}; para críticos, absoluta de 10^{-8}. Se comparan todas las filas de cada secuencia, no solo una muestra inicial.

5.8. Estimación de π y medición de tiempos

La página 6 de la guía solicita π estimado y error, pero no define el algoritmo. El proyecto adopta Monte Carlo por cuarto de círculo: cada punto es (U_{2i-1}, U_{2i}) y se cuenta como interior si $x^2 + y^2 \leq 1$. Esta es una decisión documentada del proyecto, no una fórmula atribuida a la guía.

$$\pi \text{ estimado} = 4 \cdot \text{puntos interiores} / N$$

$$\text{Error absoluto} = |\pi \text{ estimado} - \pi|$$

Aquí N significa puntos y se consumen **2N valores LCG**. Cada medición reinicia la misma semilla. Se realizan tres calentamientos de hasta 10 000 puntos y cinco repeticiones; se informa su mediana en segundos. Se mide generación más estimación, excluyendo consola, archivos y gráficas. Los relojes son perf_counter y System.nanoTime. Estas repeticiones miden rendimiento; no son muestras aleatorias independientes.

6. Resultados Obtenidos

6.1. Secuencias y estadística descriptiva

Cada lenguaje generó 10 000 valores por configuración. La tabla resume Python; los resultados Java coinciden dentro de las tolerancias declaradas. La referencia uniforme continua tiene media 0.5 y varianza 1/12 ≈ 0.08333333.

Configuración	N	Media	Varianza	Mínimo	Máximo
MINSTD	10 000	0.50128938	0.08357650	0.00025932	0.99997406
glibc	10 000	0.50210774	0.08359768	0.00001029	0.99997773
ZX81	10 000	0.50410034	0.08322764	0.00015259	0.99996948

Tabla 1. Resumen de las tres muestras. La varianza es poblacional, con divisor N. Datos: resultados/python/<generador>/analisis.json.

Las medias se encuentran entre 0.50128938 y 0.50410034, y las varianzas próximas a 1/12. Estas aproximaciones descriptivas no sustituyen las pruebas de hipótesis. En cada muestra se observaron 10 000 estados distintos. No se detectó repetición en ese tramo: **no se afirma el período total**.

La siguiente tabla muestra los diez primeros valores de MINSTD en ambos programas. Los estados se presentan completos; los normalizados se redondean únicamente para lectura.

i	X Python	X Java	U Python	U Java
1	2074941799	2074941799	0.9662200697	0.9662200697
2	559872160	559872160	0.2607107909	0.2607107909
3	1645535613	1645535613	0.7662622322	0.7662622322
4	1222641625	1222641625	0.5693368733	0.5693368733
5	1814256879	1814256879	0.8448291942	0.8448291942
6	95061600	95061600	0.0442665071	0.0442665071
7	2119961479	2119961479	0.9871839918	0.9871839918
8	1291390176	1291390176	0.6013504121	0.6013504121
9	1924951450	1924951450	0.8963753706	0.8963753706
10	817878095	817878095	0.3808541668	0.3808541668

Tabla 2. Comparación visible de los primeros diez datos MINSTD. La comprobación completa incluye también glibc y ZX81: 30 000 pares de estados y valores normalizados, sin diferencias.

Los archivos secuencia_minstd.csv, secuencia_glibc.csv y secuencia_zx81.csv de resultados/comparacion conservan la comparación fila por fila. Las secuencias originales se encuentran en resultados/python y resultados/java. La vista de consola muestra una previsualización de diez datos.

6.2. Contrastes estadísticos: N = 10 000 y α = 0.05

Chi-cuadrado: diez intervalos, $E_i = 1000$ y gl = 9.

Configuración	χ^2	p	Crítico	Decisión
MINSTD	3.2780	0.9522462	16.91897760	No se rechaza H_0
glibc	2.4360	0.9825554	16.91897760	No se rechaza H_0
ZX81	7.8600	0.5483142	16.91897760	No se rechaza H_0

Tabla 3. Prueba de uniformidad por frecuencias. Los tres valores p superan 0.05 y los tres estadísticos son menores que el crítico.

Kolmogorov-Smirnov bilateral

Configuración	D^+	D^-	D	p aprox.
MINSTD	0.004275988	0.005638671	0.005638671	0.9082184
glibc	0.001960977	0.007133834	0.007133834	0.6888559
ZX81	0.000772830	0.011633098	0.011633098	0.1334899

Tabla 4. KS asintótico. Crítico común: D = 0.01358099. No se rechaza H_0 en las tres muestras.

ZX81 presenta la mayor distancia D observada (0.011633098), aunque permanece por debajo del crítico. La diapositiva de Semana 6 presenta para glibc $p_{KS} \approx 0.686$; aquí se obtuvo 0.6888559 con el método asintótico declarado. Esa diferencia numérica no cambia la decisión y no se modificaron los datos para reproducir una cifra prefijada.

Rachas respecto a la mediana

Configuración	n_1 / n_2	Rachas R	Z	p
MINSTD	5000 / 5000	4981	-0.4000200	0.6891418
glibc	5000 / 5000	5008	0.1400070	0.8886545
ZX81	5000 / 5000	5030	0.5800290	0.5618951

Tabla 5. No hubo empates con la mediana. $\mu_R = 5001$, $\sigma^2_R = 2499.749975$ y crítico $|Z| = 1.95996398$. No se rechaza el orden aleatorio de signos en estas muestras.

Las pruebas no detectan evidencia suficiente contra sus hipótesis de referencia a α = 0.05. Un valor p mayor no constituye una puntuación de calidad del generador. Uniformidad marginal, orden de signos e independencia completa son propiedades diferentes; estos resultados no garantizan idoneidad para cualquier simulación.

6.3. Frecuencias observadas y contribuciones a χ^2

Cada intervalo incluye su extremo inferior y excluye el superior. Como U < 1, los diez intervalos cubren todos los valores generados. La frecuencia esperada es 1000 en cada intervalo y configuración.

Intervalo	MINSTD O_i	glibc O_i	ZX81 O_i	E_i
[0.0, 0.1)	978	1001	968	1000
[0.1, 0.2)	1038	973	994	1000
[0.2, 0.3)	976	989	1015	1000
[0.3, 0.4)	987	1015	947	1000
[0.4, 0.5)	991	1005	970	1000
[0.5, 0.6)	1001	984	1039	1000
[0.6, 0.7)	1019	1017	1024	1000
[0.7, 0.8)	995	985	1024	1000
[0.8, 0.9)	1011	1022	1007	1000
[0.9, 1.0)	1004	1009	1012	1000
Total	10 000	10 000	10 000	10 000

Tabla 6. Frecuencias de los CSV exportados, verificadas entre ambos lenguajes. Cada columna de conteos suma exactamente 10 000.

Intervalo	Aporte MINSTD	Aporte glibc	Aporte ZX81
[0.0, 0.1)	0.484	0.001	1.024
[0.1, 0.2)	1.444	0.729	0.036
[0.2, 0.3)	0.576	0.121	0.225
[0.3, 0.4)	0.169	0.225	2.809
[0.4, 0.5)	0.081	0.025	0.900
[0.5, 0.6)	0.001	0.256	1.521
[0.6, 0.7)	0.361	0.289	0.576
[0.7, 0.8)	0.025	0.225	0.576
[0.8, 0.9)	0.121	0.484	0.049
[0.9, 1.0)	0.016	0.081	0.144
Suma χ^2	3.278	2.436	7.860

Tabla 7. Cada aporte es $(O_i - 1000)^2/1000$. Las sumas reproducen los estadísticos de la tabla 3.

En MINSTD el mayor aporte corresponde a [0.1,0.2), con 1038 datos y aporte 1.444. En glibc corresponde a [0.1,0.2), con 973 y aporte 0.729. En ZX81 corresponde a [0.3,0.4), con 947 y aporte 2.809. Ninguna de estas desviaciones, agregada con las demás, produce rechazo al nivel fijado.

6.4.1. Gráficas de MINSTD

El histograma presenta conteos próximos a 1000. La FDE se mantiene cerca de la diagonal y su mayor separación es D = 0.005638671, inferior al crítico. La coincidencia visual con la uniforme es consistente con las pruebas; no demuestra independencia.

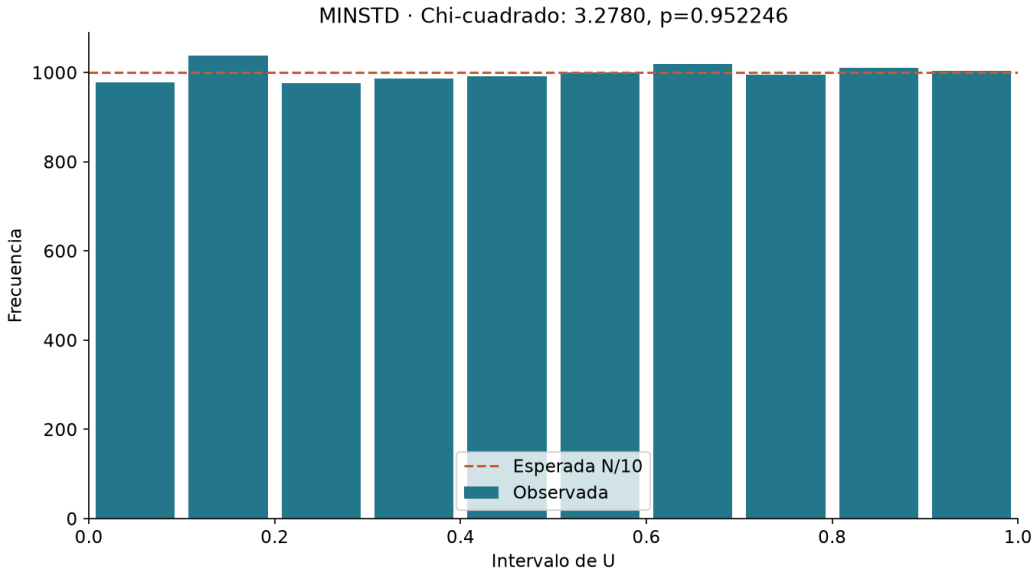


Figura 1. MINSTD: frecuencias observadas frente a E_i = N/10. Gráfica real exportada por la implementación Python.

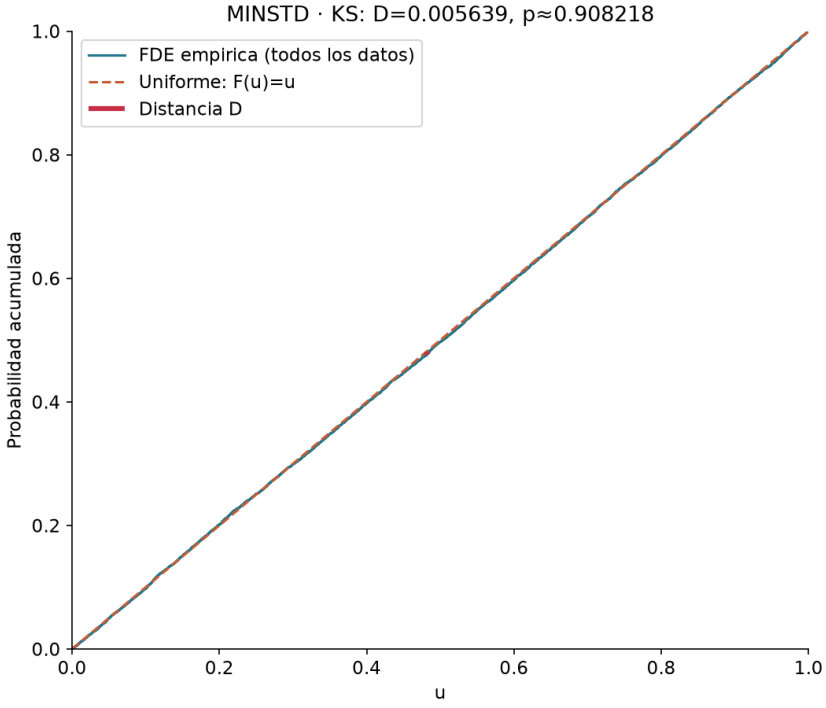


Figura 2. MINSTD: FDE de los 10 000 datos frente a F(u)=u, con la distancia máxima D. Se representa la secuencia completa ordenada.

Java también conserva histograma.png y acumulada.png en su carpeta de resultados. Los cálculos usados en ambas representaciones se contrastaron antes de elaborar las tablas.

6.4.2. Gráficas de glibc

Las frecuencias varían entre 973 y 1022. La mayor distancia entre la FDE y F(u)=u es D = 0.007133834. La tabla 4 informa ambas diferencias; la línea roja señala la distancia D. Estos resultados corresponden a la configuración LCG de clase y a la semilla 1.

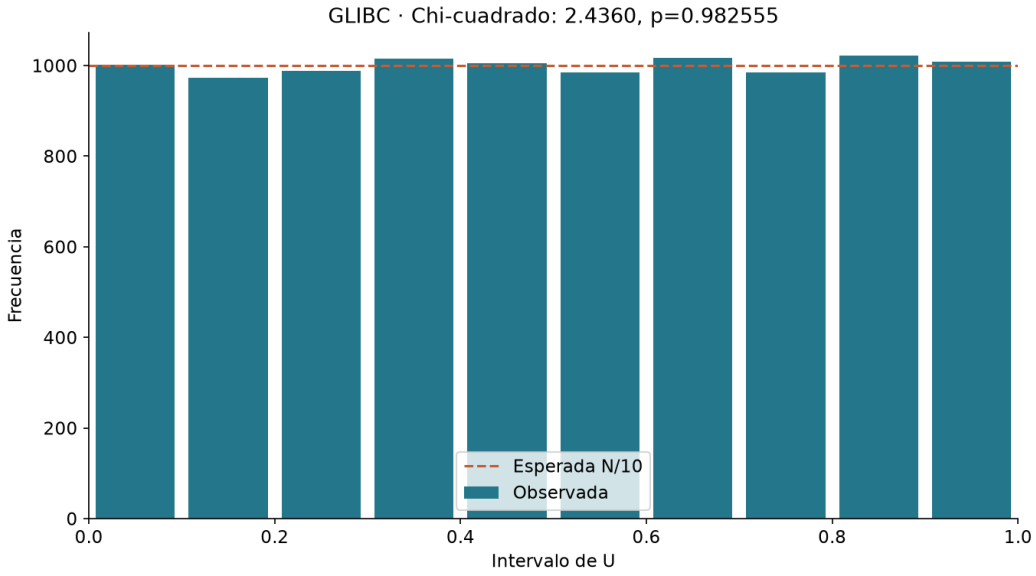


Figura 3. glibc: frecuencias observadas frente a $E_i = N/10$. Gráfica real exportada por la implementación Python.

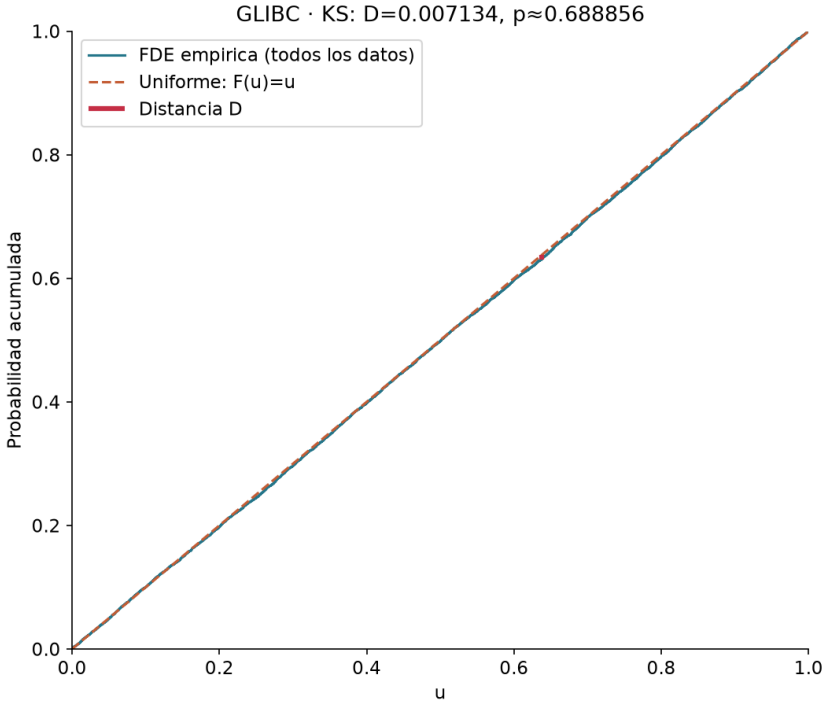


Figura 4. glibc: FDE de los 10 000 datos frente a F(u)=u, con la distancia máxima D. Se representa la secuencia completa ordenada.

Java también conserva histograma.png y acumulada.png en su carpeta de resultados. Los cálculos usados en ambas representaciones se contrastaron antes de elaborar las tablas.

6.4.3. Gráficas de ZX81

Las frecuencias varían entre 947 y 1039 y la distancia máxima es D = 0.011633098. Aunque ZX81 se identifica como deficiente en la tabla de Semana 5, estas pruebas con N = 10 000 no lo rechazan. No se fuerza un resultado distinto; el alcance de la conclusión es esta muestra.

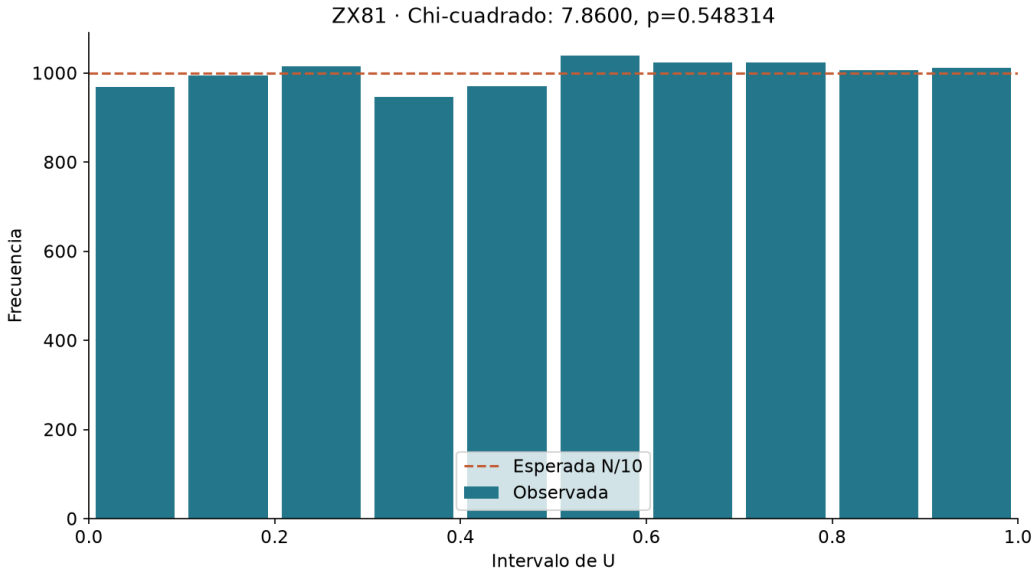


Figura 5. ZX81: frecuencias observadas frente a E_i = N/10. Gráfica real exportada por la implementación Python.

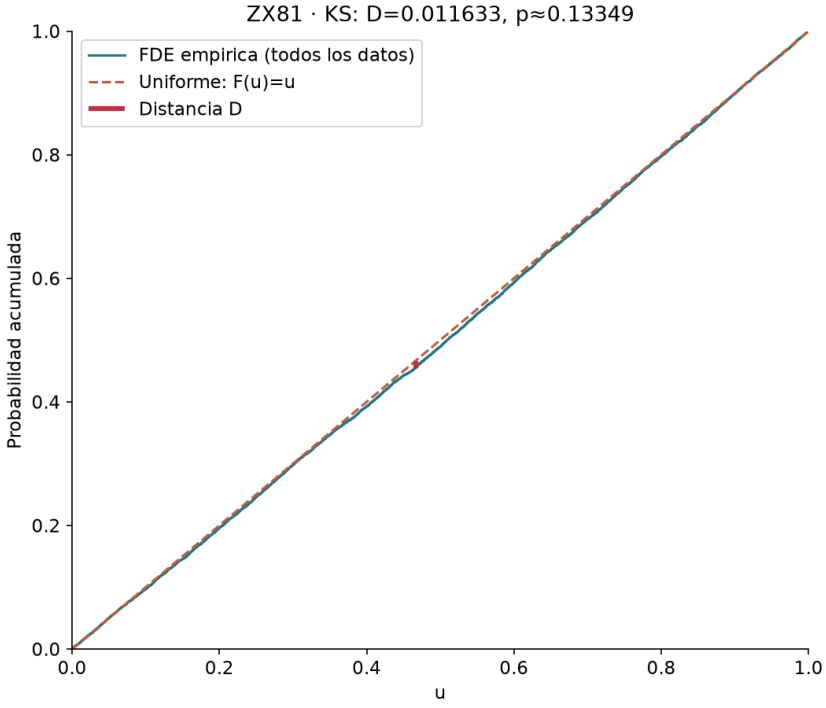


Figura 6. ZX81: FDE de los 10 000 datos frente a F(u)=u, con la distancia máxima D. Se representa la secuencia completa ordenada.

Java también conserva histograma.png y acumulada.png en su carpeta de resultados. Los cálculos usados en ambas representaciones se contrastaron antes de elaborar las tablas.

6.5. Inspección visual de pares consecutivos

Se muestran los primeros 4000 pares (U_n, U_{n+1}) en Python y Java. Las escalas y estilos gráficos difieren; los datos de las secuencias coinciden. Esta inspección complementa las pruebas marginales y no se presenta como una prueba serial formal.

MINSTD — Python (izquierda) y Java (derecha)

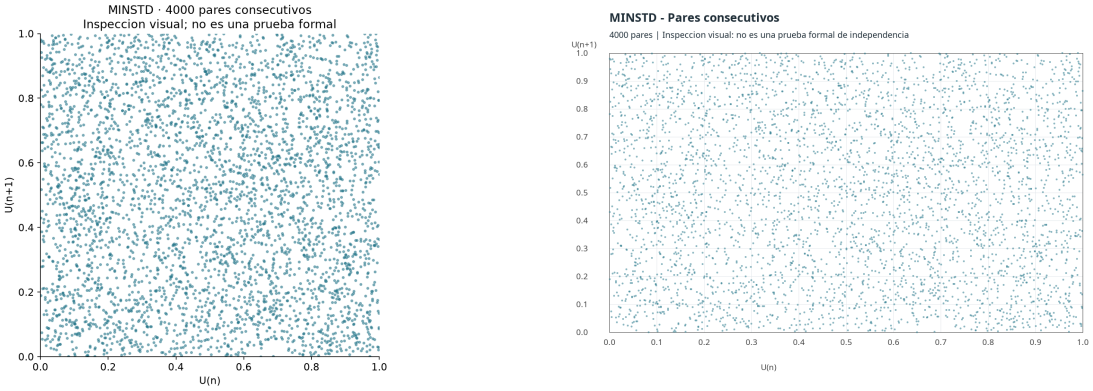


Figura 7. Archivos pares.png generados independientemente por ambos programas.

glibc — Python (izquierda) y Java (derecha)

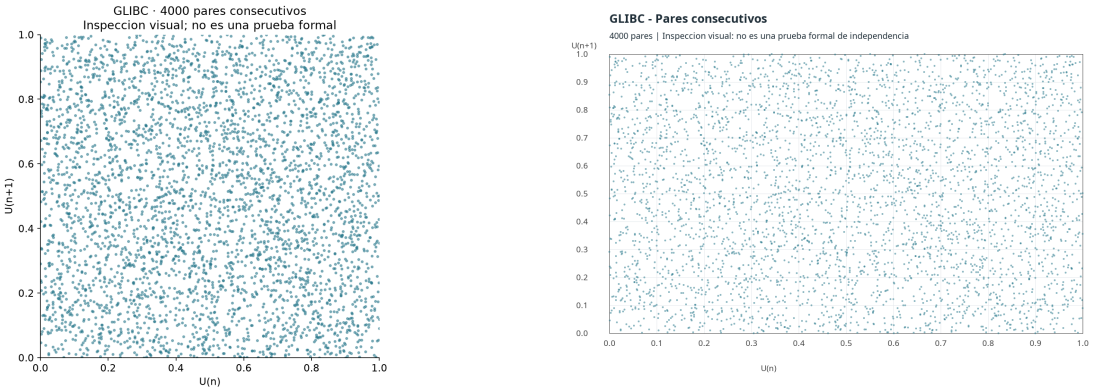


Figura 8. Archivos pares.png generados independientemente por ambos programas.

ZX81 — Python (izquierda) y Java (derecha)

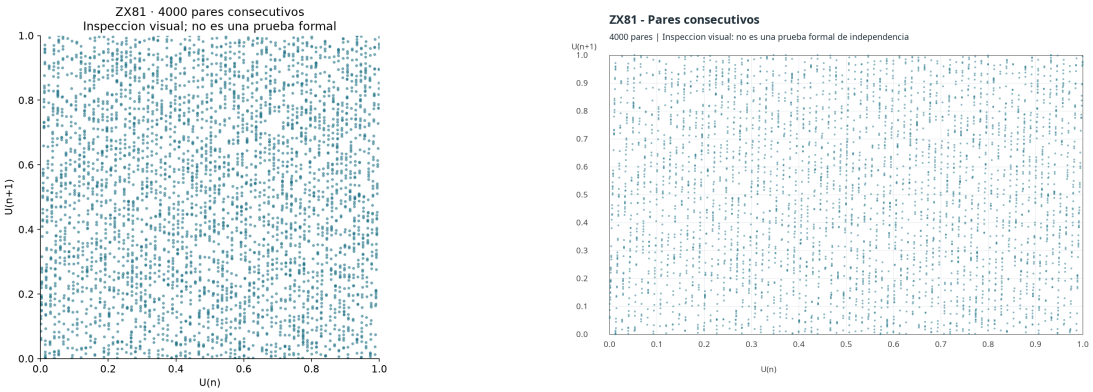


Figura 9. Archivos pares.png generados independientemente por ambos programas.

Que una nube parezca uniforme no descarta estructuras en otras dimensiones o retardos. No se ejecutaron poker, una prueba serial formal, Diehard ni TestU01.

6.6. Estimación de π, error y tiempos de ejecución

La tabla reproduce los cuatro tamaños de la guía para las tres configuraciones. Las estimaciones y los conteos interiores coinciden entre lenguajes; por eso se muestra una columna de π. Los tiempos son las medianas de cinco mediciones y corresponden a los archivos benchmark.csv conservados.

Generador	N puntos	π estimado	Error absoluto	Python (s)	Java (s)
MINSTD	1 000	3.15200000	0.01040735	0.000261	0.000064
MINSTD	10 000	3.14920000	0.00760735	0.003079	0.000496
MINSTD	100 000	3.14068000	0.00091265	0.029200	0.001534
MINSTD	1 000 000	3.14322400	0.00163135	0.301097	0.012087
glibc	1 000	3.14400000	0.00240735	0.000303	0.000015
glibc	10 000	3.14240000	0.00080735	0.003086	0.000113
glibc	100 000	3.14556000	0.00396735	0.031545	0.001262
glibc	1 000 000	3.14193600	0.00034335	0.304008	0.011903
ZX81	1 000	3.12800000	0.01359265	0.000181	0.000016
ZX81	10 000	3.12360000	0.01799265	0.001902	0.000145
ZX81	100 000	3.12272000	0.01887265	0.018836	0.001366
ZX81	1 000 000	3.12299600	0.01859665	0.190781	0.011684

Tabla 8. Error = |π estimado−π|; tiempos en segundos, redondeados para lectura. N puntos consumen 2N uniformes. Los CSV conservan los valores completos y tiempos.csv las 120 mediciones individuales.

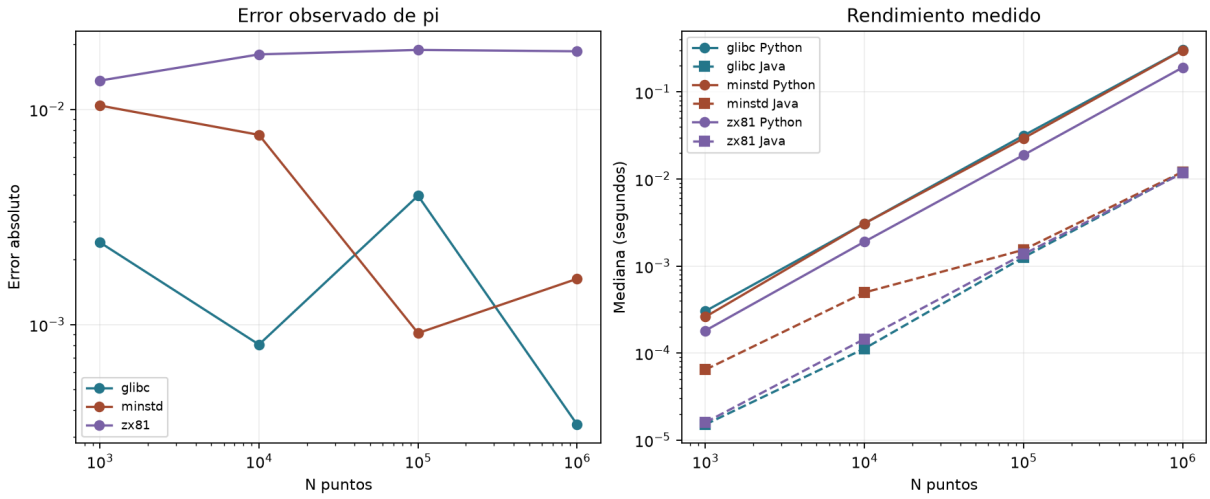


Figura 10. Error absoluto y medianas de tiempo de los mismos datos de la tabla 8. Ambos ejes usan escalas logarítmicas.

Con un millón de puntos, glibc obtuvo el menor error observado (0.00034335), seguido de MINSTD (0.00163135) y ZX81 (0.01859665). El error no disminuye monótonamente: en glibc aumenta de N = 10 000 a 100 000, y en MINSTD de 100 000 a 1 000 000. Una única secuencia no establece una clasificación universal.

Para N = 1 000 000, las razones tiempo Python / tiempo Java fueron 24.91 en MINSTD, 25.54 en glibc y 16.33 en ZX81. Java fue más rápido en estas mediciones. La conclusión se limita a estas implementaciones, relojes y entorno; no es una comparación universal de lenguajes.

6.7. Verificación computacional y trazabilidad

La verificación existente del proyecto registra las siguientes pruebas. Los extractos inferiores son texto de los registros reales, no capturas de una interfaz ni resultados de pruebas añadidas para este informe.

Suite	Ejecutadas	Fallos	Errores	Omitidas
Python / pytest	32	0	0	0
Java / Maven y JUnit	19	0	0	0

Tabla 9. Evidencia: resultados/revision/pytest.log y maven.log. Maven también registra BUILD SUCCESS.

```
.............................                                        [100%]
32 passed in 5.20s
```

```
[INFO] Tests run: 19, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
[INFO] Finished at: 2026-11-08T16:05:14-05:00
```

Las pruebas incluyen vectores LCG conocidos, ciclos pequeños calculables a mano, límites de intervalos, datos inválidos, referencias de SciPy, rachas, estimación de π y una integración Python/Java con parámetros de 48 bits. La integración altera deliberadamente un estado de una copia temporal para comprobar que la comparación detecta la discrepancia. Esto no modifica las secuencias entregadas.

Antes de generar este PDF se repitió la comparación de las secuencias existentes en una carpeta temporal. También se exigieron explícitamente N = 10 000 y α = 0.05, las tres configuraciones y los cuatro tamaños de benchmark. Se comprobaron las frecuencias exportadas, la longitud de las tablas KS y las medianas y mínimos frente a las cinco mediciones individuales de cada caso.

Evidencia comprobada	Resultado
Secuencias Python frente a Java	30 000 pares de estados y valores normalizados iguales
Pruebas estadísticas	Nueve resultados por lenguaje; decisiones coincidentes
Tabla de π	12 estimaciones y conteos interiores coincidentes
Tiempos individuales	60 por lenguaje; 120 en total; medianas y mínimos coherentes
Artefactos gráficos de cada programa	18 PNG: histograma, acumulada y pares por configuración y lenguaje

Tabla 10. La coincidencia entre implementaciones acredita reproducción del cálculo, no azar verdadero ni independencia completa. La instalación en un equipo diferente no se presenta como verificada.

7. Preguntas de Control

1. ¿Qué es la hipótesis nula?

Es la afirmación de referencia que se somete a contraste. Aquí, H_0 establece que la distribución de los valores es U(0,1). En rachas, la referencia es el orden aleatorio de los signos respecto a la mediana.

2. ¿Qué representa el valor p?

Bajo H_0 y los supuestos de la prueba, es la probabilidad de obtener un estadístico tan extremo o más que el observado. No es la probabilidad de que H_0 sea verdadera.

3. ¿Qué significa rechazar H_0?

Significa que los datos aportan evidencia contra H_0 al nivel de significancia elegido. Con $\alpha = 0.05$ se rechaza cuando $p < 0.05$. Puede ocurrir un error tipo I (rechazar una H_0 verdadera); no rechazar una falsa es un error tipo II.

4. ¿Qué diferencia existe entre Chi-cuadrado y KS?

Chi-cuadrado agrupa los valores y compara frecuencias observadas y esperadas; depende de los intervalos. KS compara la FDE de los datos individuales ordenados con F(u)=u mediante la distancia máxima D.

5. ¿Por qué una prueba estadística no demuestra que un generador sea verdaderamente aleatorio?

El LCG es determinista y reproducible a partir de su semilla. Cada prueba detecta aspectos concretos; superar una batería limitada no descarta otros patrones. Por ejemplo, ZX81 no fue rechazado aquí, aunque su error de π con un millón de puntos fue mayor que el de las otras configuraciones.

8. Conclusiones

1. Se implementó la misma recurrencia en Python y Java. Coincidieron exactamente los 30 000 estados y valores normalizados comparados, y las doce estimaciones de π; la ejecución es reproducible con los parámetros usados.

2. A $\alpha = 0.05$, Chi-cuadrado y KS no rechazaron uniformidad en ninguna de las tres muestras. Rachas tampoco rechazó el orden aleatorio de signos. Estas decisiones se limitan a las pruebas y N = 10 000.

3. Con un millón de puntos, los errores absolutos de π fueron 0.00163135 (MINSTD), 0.00034335 (glibc) y 0.01859665 (ZX81). La mayor desviación de ZX81 muestra por qué superar pruebas marginales no garantiza el mismo desempeño en una aplicación. No se atribuye a una única causa sin análisis adicional.

4. Java registró menores tiempos en la tabla conservada. La medición excluyó archivos y gráficas; las diferencias corresponden a este entorno y a estas implementaciones. El método de π, el error absoluto y el carácter asintótico de KS se declararon explícitamente para delimitar las conclusiones.

9. Bibliografía y fuentes de la práctica

[1] José O. Guamán Q. *Guía de Actividades Práctico-Experimentales Nro. 006: Validación estadística de números pseudoaleatorios mediante Chi-cuadrado y Kolmogorov-Smirnov.* Simulación, U2. 05 y 06 de noviembre de 2026. Archivo: ape_guia simulacion semana 6.pdf; pp. 1–6.

[2] Material de la asignatura. *Generadores pseudoaleatorios, Semana 5.* Archivo: simulacion Semana 5.pptx [Repaired].pdf. Se consultaron las pp. 7–10 (recurrencia y período), 13 (parámetros), 15 (pares consecutivos) y 19 (ejemplo MINSTD). Ubicación: Simulacion/Diapos.

[3] Material de la asignatura. *Semana 6: Pruebas estadísticas de uniformidad e independencia.* Archivo PDF del mismo nombre. Se consultaron las pp. 6–11 (uniformidad), 13–14 (rachas), 18 (ejemplo glibc) y 19–25 (interpretación y límites). Ubicación: Simulacion/Diapos.

[4] Proyecto 06APE. Código Python y Java, config/generadores.csv, archivos de resultados, registros de verificación y docs/BASE_ACADEMICA.md. Fuentes directas de las cifras de este reporte.

[5] Anderson Mateo Coello Jaramillo. *Reportes técnicos de las prácticas 001 y 002, Unidad 1.* Archivos APE01-CoelloAnderson-U1-Simulacion.pdf y APE02-CoelloAnderson-U1-Simulacion.pdf; y reporte APE04-CoelloAnderson-U1-Simulacion.pdf. Referencias de formato, encabezado e identificación. Sus resultados experimentales no se trasladaron al APE 06.

10. Anexos y reproducción de resultados

Desde la carpeta 06APE, la preparación, ejecución completa y verificación se realizan con:

```
./scripts/preparar.sh
./scripts/ejecutar_todo.sh
./scripts/verificar.sh
```

Para regenerar este informe institucional a partir de resultados ya existentes y verificados:

```
python/.venv/bin/python scripts/generar_informe.py
```

Archivo / ubicación	Contenido
config/generadores.csv	Semillas y parámetros compartidos.
resultados/{python,java}/<generador>/	Secuencia completa, frecuencias, datos KS, análisis JSON y gráficas PNG.
resultados/{python,java}/benchmark.csv y tiempos.csv	Resultados de π, medianas y mediciones individuales.
resultados/comparacion/	Secuencias contrastadas, tabla_guia.csv y verificacion.json.
resultados/revision/	Registros de pytest, Maven y comparación.

Informe elaborado el 08/11/2026 (America/Guayaquil). Las fechas de la tabla de identificación corresponden al calendario de la práctica indicado por la guía. Regenerar el PDF no vuelve a medir los tiempos; para obtener otras mediciones se debe ejecutar nuevamente el benchmark.